TRAINING ON NSF ACCESS

From your lab to a supercomputer

- **NSF ACCESS** = free, proposal-based GPU time on clusters aggregated across nation-wide universities — DeltaAI (UIUC), Bridges-2 (CMU), Anvil (Purdue), Expanse (UCSD).
- Assumed mode: simple imitation learning policy (ACT) — 232 episodes up, **~1.5 GPU-hours**.
- Get on ACCESS, then **seven steps**

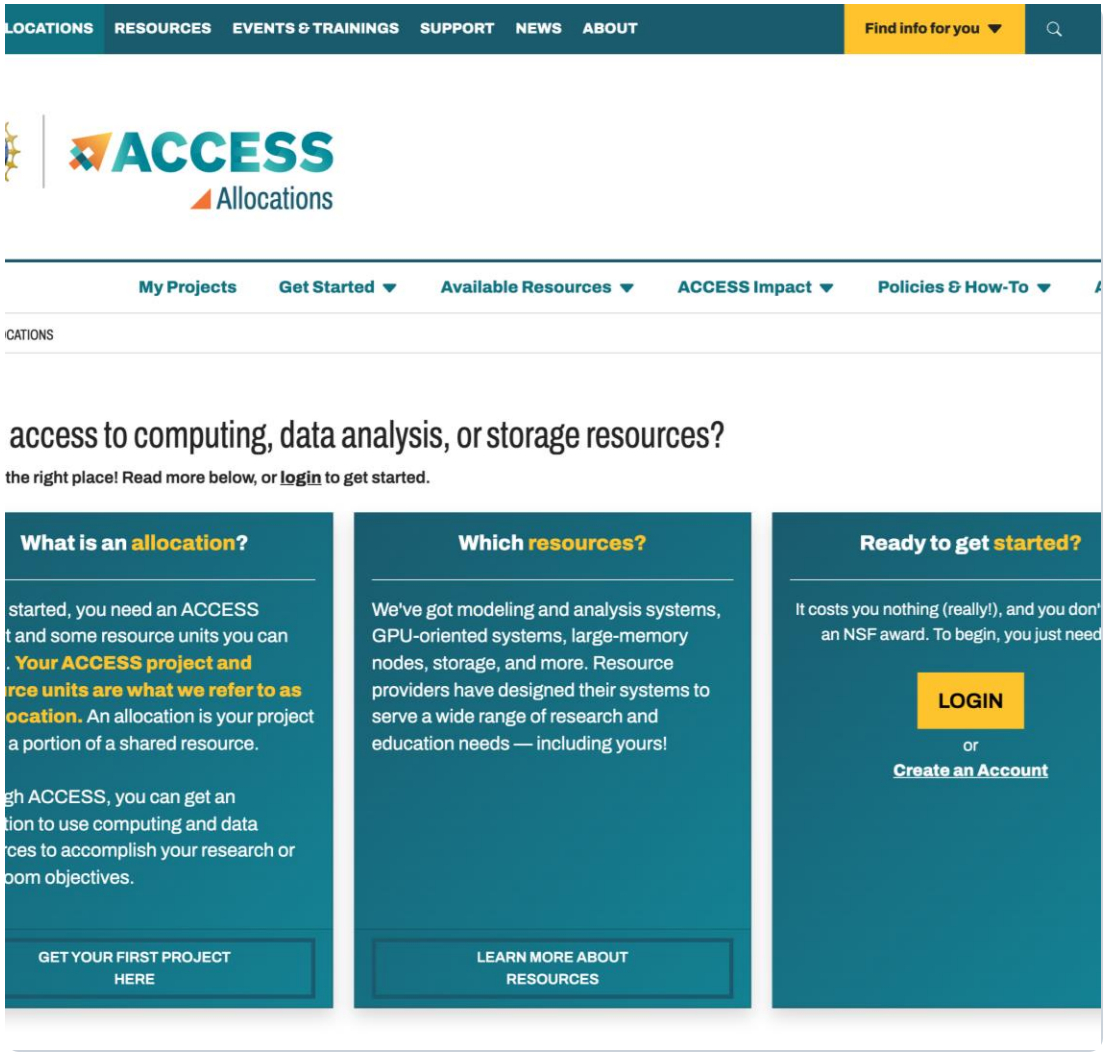
Free national compute, by proposal

→ **What it is.** An NSF program that allocates time on national supercomputers to US academics — no cost, you apply with a short proposal.

→ **The currency.** You're granted **ACCESS Credits** , then exchange them for GPU-hours on one machine (we use NCSA **DeltaAI**).

→ **The machines.** DeltaAI & Delta (NCSA), Bridges-2 (PSC), Anvil (Purdue), Expanse (SDSC) — pick by GPU type and availability.

→ **The tiers.** Explore → Discover → Accelerate → Maximize. A lab ML project fits in **Explore** (cap 400k credits); ours used ~1.5% of a small grant.



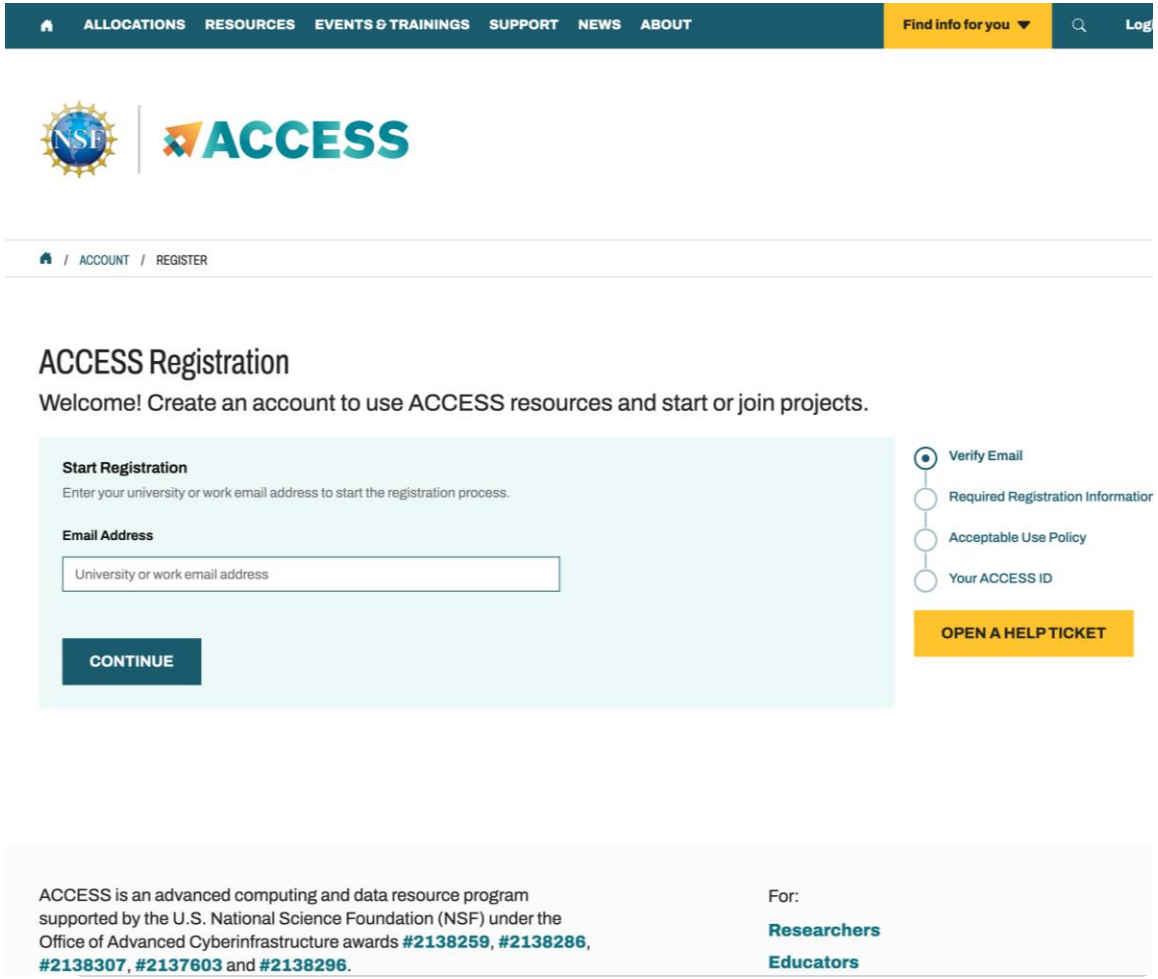
The ACCESS Allocations portal — `allocations.access-ci.org`

STEP 0 · GET ON ACCESS — CREATE AN ACCOUNT

Register your ACCESS ID, then propose

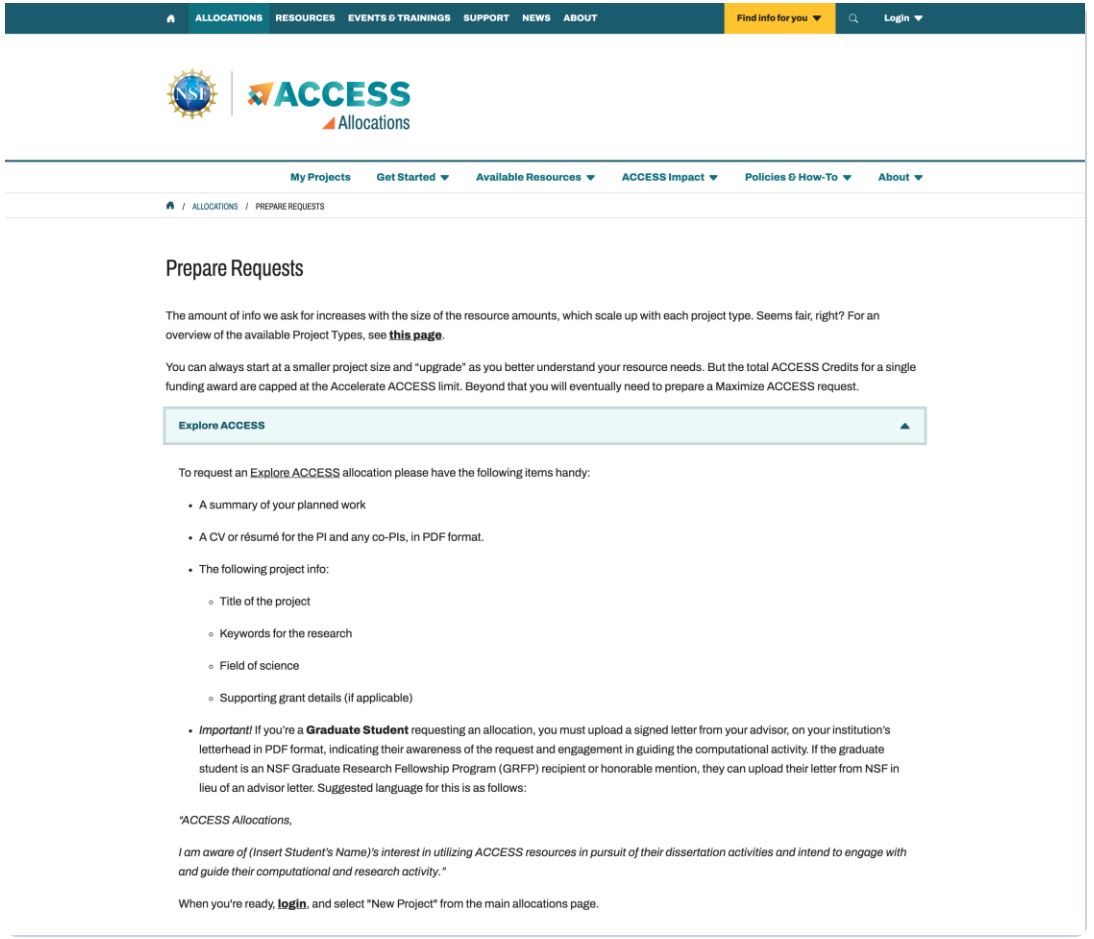
1 Registe

Create an **ACCESS ID** at `operations.access-ci.org` , verifying your university email. Sign in through **CILogon** and enroll **Duo MFA**.



2 Propose (Explore)

At `allocations.access-ci.org` → **Prepare Requests**. Explore needs a **work summary + PI/co-PI CV (PDF)** — no merit panel, ~next-business-day. Your PI / lead (William) owns it.



Short Proposal Template

→ **Fairly perfunctory. Get Nikolaus to sign on DocuSign and submit.**
Link: https://www.overleaf.com/project/6947568c72aae0ed99ee753f

NSF ACCESS Program
Confirmation letter for William Xie

Dear Colleagues,

I am writing today in support of William Xie's proposal for an ACCESS application for his research work on training models for contact-rich robotic manipulation and whole body control. William has been working on research with me since the beginning of September 2023, the start of his PhD, and I serve as his supervisor.

This project has two aims. First, we are developing new models to predict and control contact at the fingertips, wrist, and arm joints of a six degree-of-freedom (DoF) robotic arm and 27-DoF humanoid platform, both equipped with parallel jaw grippers and dexterous 6-DoF hands with sensorized fingers. Current models, such as vision-language-action (VLA), leverage internet-scale pretraining to provide positional robot commands to perform a number of tasks, but have not proven capable of dexterous, complex manipulation tasks requiring abundant, continuous contact with the external world. Second, we are working on sensorizing the entire bodies of our robots with a tactile skin and are investigating the best learning methods for integrating this rich sensory data into whole body control policies.

I am in full support of William's work on this project.

Please feel free to contact me with any questions!

Sincerely,

Nikolaus Correll
University of Colorado at Boulder

The ACCESS Allocations portal — `allocations.access-ci.org`

STEP 0 · EXCHANGE CREDITS → DELTAAI HOURS

Turn credits into GPU-hours (200,000 credits, auto-approved for 200,000 more)

Number of units on this resource:

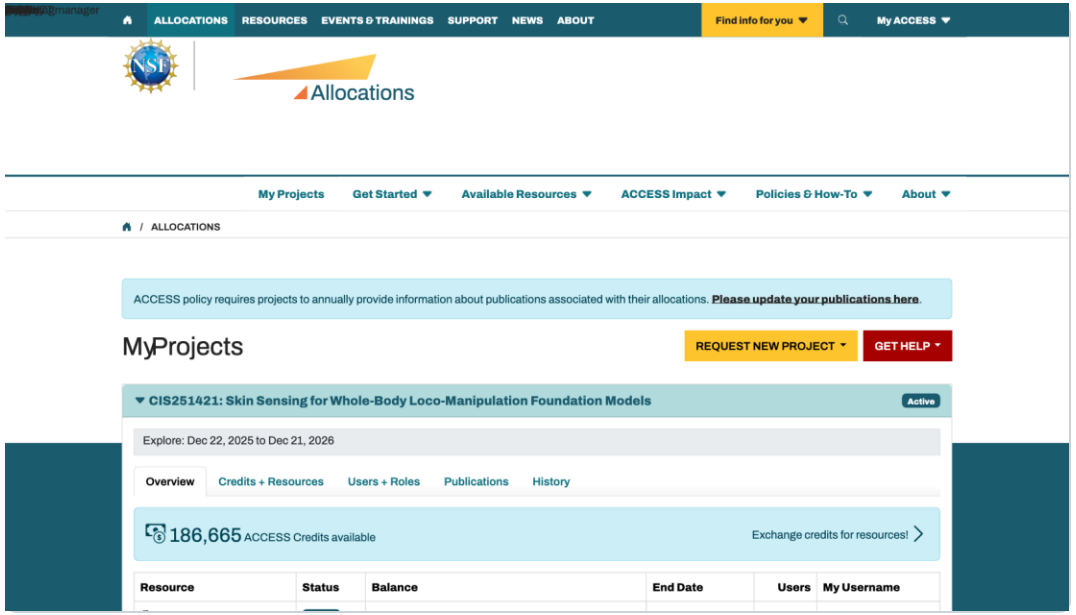
1,000	ACCESS Credits

Exchange Rates Overview:

Amount	Unit Type	Target System
1,000	ACCESS Credits	ACCESS Credits
1,000	Number of Students	CloudBank Classroom
25	Dollars	CloudBank Research
8,000	Service Units	IACS at Stony Brook Ookami
1,000	Service Units	Indiana Jetstream2 CPU
1,000	Service Units	Indiana Jetstream2 GPU
1,000	Service Units	Indiana Jetstream2 Large Memory
1,000	GigaBytes	Indiana Jetstream2 Storage
1,001	Core-hours	Kentucky Research Informatics Cloud (KyRIC) Large Memory Nodes
7	GPU Hours	NCSA DeltaAI
1,000	Core-hours	NCSA Delta CPU
15	GPU Hours	NCSA Delta GPU

Credit Exchange & Calculator — NCSA DeltaAI row
https://allocations.access-ci.org/exchange_calculator

Rate	**1,000 credits → 7 DeltaAI GPU-hours** (~143 credits / GPU-hr)
Charge unit	1 GPU-hour = **one GH200 for one hour**
Whole node	node-exclusive = **4 GH200-hrs per wall-clock hour**
Billed on	resources **reserved** , not used — request only what you need



Dashboard confirms the live allocation — **186,665 credits · DeltaAI 97/1400 GPU-hrs**

HOW IT WORKS · THE MENTAL MODEL + YOUR COORDINATES

Four machines, one scheduler

Your machine
Hold the data · develop · deploy the policy

↓ `rsync` data up (Duo once)

Login node — shared, many users
Set up env · smoke-test · `sbatch`. **DO NOT train here.**

↓ **Slurm** (batch scheduler) queues the job, runs it when your GPU/CPU/RAM/time are free

Compute node — 1× GH200, yours alone
The training actually happens here, unattended

↓ `rsync` the trained model back down

Your machine
Load, verify, run the policy on the robot

Relevant user and training information — fill these in first

Login host	`dtai-login.delta.ncsa.illinois.edu`
Username	`aabid` (ACCESS mapping)
Slurm account	`bgcd-dtai-gh` (run accounts)
GPU partition	`ghx4`
CPU arch	**`aarch64`** — ARM

> **Why ARM?** A GH200 fuses a 72-core Arm *Grace* CPU to a *Hopper* GPU. The CPU is `aarch64`, so every Python wheel must be the ARM build — the root of most setup pain.

`bgcd-dtai-gh` = your **Slurm account** — the project every GPU-hour bills against. Run `accounts` after login to print yours.

`ghx4` = DeltaAI's GPU partition: a named pool of nodes, each with 4× GH200. `--partition=ghx4` tells Slurm to place your job there.

The DeltaAI user guide — `docs.ncsa.illinois.edu/systems/deltaai` — plus `accounts`, `sinfo`, and `uname -m` run on the login node.

STEP 1 · LOG IN — ONCE, THEN NEVER RE-AUTH

One Duo prompt for the whole session

```
# ~/.ssh/config

Host deltaai
    HostName        dtai-login.delta.ncsa.illinois.edu
    User            <YOUR_USERNAME>
    ControlMaster   auto
    ControlPath     ~/.ssh/cm-%r@%h-%p
    ControlPersist  12h
```

`ssh deltaai exit` opens the connection — password + Duo **once**.

What each line does

`ControlMaster auto`	first connection becomes a master others ride on
`ControlPath`	on-disk socket file later commands attach to
`ControlPersist 12h`	keep it alive 12 h after logout → overnight jobs need no human

→ Once connected, run `accounts` — it prints your Slurm account (e.g. `bgcd-dtai-gh`) to put in `--account`.

STEP 2 · PUSH YOUR DATA UP

Move the dataset to the cluster

```
rsync -avz --progress ~/magpie_control/data/lerobot_v0/ deltaai:~/lerobot_v0/
```

→ left path = the dataset on your machine · right path = where it lands on the cluster

Decode the command

`-a`	archive: recurse, preserve permissions, timestamps, symlinks
`-z`	compress in transit
`-v --progress`	list files and show a live transfer bar
`trailing /`	copy the *contents* of the folder, not the folder itself

Incremental. rsync compares both sides and sends only new/changed files — rerun anytime. Anecdata: 60-episodes (387 MB) went up in ~90 s; re-runs send just the delta.

Push only what's verified. Confirm the dataset is intact locally before it goes up — debugging a broken dataset through a Slurm queue costs minutes-to-hours per attempt.

STEP 3 · BUILD THE ENVIRONMENT (ONE-TIME)

Module supplies torch; venv layers the rest

```
module purge
module load python/miniforge3_pytorch/2.7.0
module load cuda
python -m venv --system-site-packages ~/lerobot_env
source ~/lerobot_env/bin/activate
pip install lerobot==0.4.4
pip uninstall -y torch torchvision torchcodec
python -c "import torch; print(torch.version.cuda)"
```

Why each line

`module purge`	clear any inherited modules so you start from a clean, known base
`module load …2.7.0`	site-maintained torch + torchvision, compiled for this GPU
`module load cuda`	adds the CUDA runtime libraries torch links against
`--system-site-packages`	lets the venv *see* the module's torch, so you never reinstall it
`source …/activate`	enter the venv — every `pip`/`python` after this uses it
`lerobot==0.4.4`	pin the exact version that *wrote* the dataset, or the format won't load
`pip uninstall torch…`	evict any CPU wheel pip dragged in as a dependency (see next slide)
`print(...cuda)`	must print **12.6**, not `None` — confirms torch sees the GPU

REFERENCE · FIRST-RUN GOTCHAS ON ARM

Four things that trip up a first run — and the fix

What you see	What it actually is	Fix
`torch.version.cuda = None`	pip installed the **CPU-only** torch wheel — the default on ARM. torch can't see the GPU, so training silently runs on CPU or crashes.	Uninstall it; use the module's torch
`VideoReader missing`	`VideoReader` is torchvision's video decoder (`torchvision.io`) — lerobot uses it to read the episode `.mp4`s. The *newest* module dropped it. Newer ≠ compatible.	Load the older 2.7.0 module
`libcufile.so.0 not found`	torch links against a CUDA runtime library that isn't on the path because the CUDA module wasn't loaded.	`module load cuda`
`Job exits at startup, before step 1`	Expected default behavior, not a bug: lerobot tries to **upload the policy to the Hugging Face Hub**. With no repo named it raises immediately — flip the flag off and it trains.	`--policy.push_to_hub=false`

STEP 4 · SMOKE-TEST THE DATA PATH

Dataloading one sample

Run this on the login node, line by line — it walks the exact path the training job takes:

1	`from lerobot.datasets... import LeRobotDataset`	import lerobot's dataset loader — the same class the trainer uses.
2	`ds = LeRobotDataset('magpie/v0', root='$HOME/lerobot_v0')`	open the dataset you pushed in Step 2 (points at the folder on the cluster).
3	`ds[0]`	fetch one sample — this **forces a real video decode** , the exact operation the job does 150k times. Wrong codec/torchvision fails *here*, in seconds.
4	`print('sample OK')`	reach this line and the data path works. No Slurm queue involved

Full command, copy-paste:

```
python -c "
from lerobot.datasets.lerobot_dataset import LeRobotDataset
ds = LeRobotDataset('magpie/v0', root='$HOME/lerobot_v0')
ds[0]; print('sample OK')
"
```

STEP 5 · THE SLURM JOB — ~/ACT_V0.SLURM

A resource request the scheduler runs for you

```
#!/usr/bin/env bash
#SBATCH --account=bgcd-dtai-gh
#SBATCH --partition=ghx4
#SBATCH --gpus-per-node=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=64G
#SBATCH --time=10:00:00
#SBATCH --output=act_v0_%j.log
set -euo pipefail
module purge
module load python/miniforge3_pytorch/2.7.0 cuda
source ~/lerobot_env/bin/activate
python -m lerobot.scripts.lerobot_train \
  --dataset.repo_id=magpie/v0 \
  --dataset.root=$HOME/lerobot_v0 \
  --policy.type=act --policy.push_to_hub=false \
  --output_dir=$HOME/act_v0_run \
  --steps=150000 --save_freq=10000
```

All of this is **one file** — save it as `~/act_v0.slurm`, submit with `sbatch`.

`#SBATCH`	a **resource request** the scheduler reads — not shell commands
`--account / --partition`	what the GPU-hours bill against; `ghx4` picks the GH200 pool
`gpus / cpus / mem`	reserve 1 GPU, 8 CPU cores, 64 GB RAM
`--time=10:00:00`	hard wall-clock cap; killed at it. Shorter caps schedule sooner
`set -euo pipefail`	abort on the first error instead of running on in a broken env
`module load …`	**repeat here** — the compute node starts from a clean shell, not your login env
`--save_freq=10000`	checkpoint often

STEP 6 · SUBMIT & MONITOR (2 MINUTES)

Submit, confirm it's really training, log out

1	**Submit**	`sbatch act_v0.slurm`	hands the script to Slurm and returns a job ID **immediately** — it does not wait for the job to run.
2	**Check**	`squeue -u $USER`	shows the state: `PD` = pending (waiting for a GPU), `R` = running. 1-GPU jobs reach R in minutes.
3	**Watch**	`tail -f act_v0_*.log`	streams the log live. **Loss lines ticking = healthy**; an instant traceback = fix now.
4	**Log out**	`Ctrl-C → exit`	`Ctrl-C` stops *watching*, not the job — it runs on the compute node after you disconnect.

> Our numbers: **0.037 s/step** on a GH200 → 150k steps ≈ **1.5 hours**. Total babysitting time: about two minutes.

STEP 7 · PULL THE MODEL HOME & VERIFY

Copy the model down and check it before deploying

All from **your local machine** — still requires open connection to cluster though (the 12-hour socket means no new Duo prompt):

1 **Copy down**	`rsync -avz deltaai:.../checkpoints/last/pretrained_model/ ~/models/act_v0/`	pull the trained policy off the cluster. `checkpoints/last` = newest step; `pretrained_model/` = `config.json` + `model.safetensors`, ~200 MB.
2 **Load**	`python -c "ACTPolicy.from_pretrained('.../act_v0')"`	open it locally to confirm it loads — a lerobot version mismatch bites *here* , on your bench, not on the robot.
3 **Validation**	`replay on held-out data`	run the policy offline against data it never trained on **before** it touches hardware.

> Our replay test measured **1.9 mm error** — we knew the policy was sound before the robot ever moved.

Full commands, copy-paste:

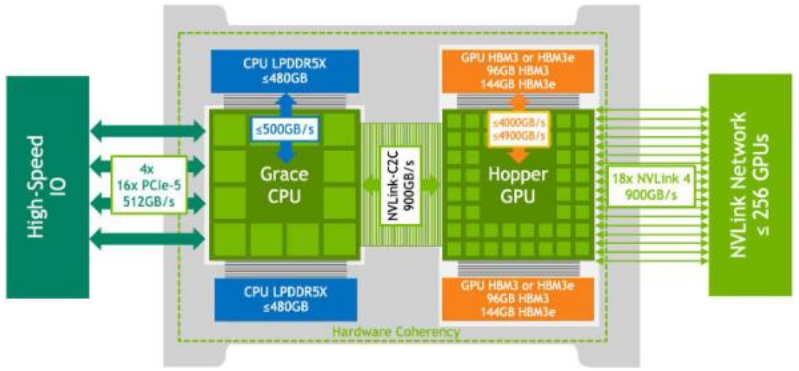
```
rsync -avz deltaai:act_v0_run/checkpoints/last/pretrained_model/ ~/magpie_control/models/act_v0/
python -c "ACTPolicy.from_pretrained('~/magpie_control/models/act_v0')"
```

WHAT 1.5 GPU-HOURS BOUGHT

DeltaAI Hardware Information

DeltaAI:
- 608 H100 NVIDIA GPUs.
- 200 Gb/s HPE SlingShot network fabric.
- Two Lustre file systems (based on HDD and NVME)
- 152 CPU-GPU nodes:
    - 4 Grace Hopper GH200 superchips per node

GPU	**NVIDIA GH200** — Hopper, H100-class, ~96 GB HBM3
CPU / arch	**aarch64 (ARM)** — 72-core Grace (120 GB of LPDDR5X memory)
Requested	1 GPU · 8 CPU · 64 GB RAM · 10 h cap
VRAM used	A few GB of ~96 — batch 8 barely dents it
Dataset	**V1: 442 MB / 232 eps** · 25,993 frames @ 10 Hz (V0: 387 MB / 60)
GPU-hours	**~1.5 of a 1400-hour** grant (~0.1%)
Throughput	0.037 s/step → 150k steps ≈ 1.5 h
Loss	ACT **L1 + KL** (CVAE), not MSE → ≈0.04
Storage	3.5 TB NVME On-Compute-Node Storage



https://nvdam.widen.net/s/rrgqqnpbz8/grace-datasheet-gh200-grace-hopper-superchip-3773000%20

https://delta.ncsa.illinois.edu/deltaai-hardware-and-network/